

2024/2025 Second Semester

SC2005

Operating Systems

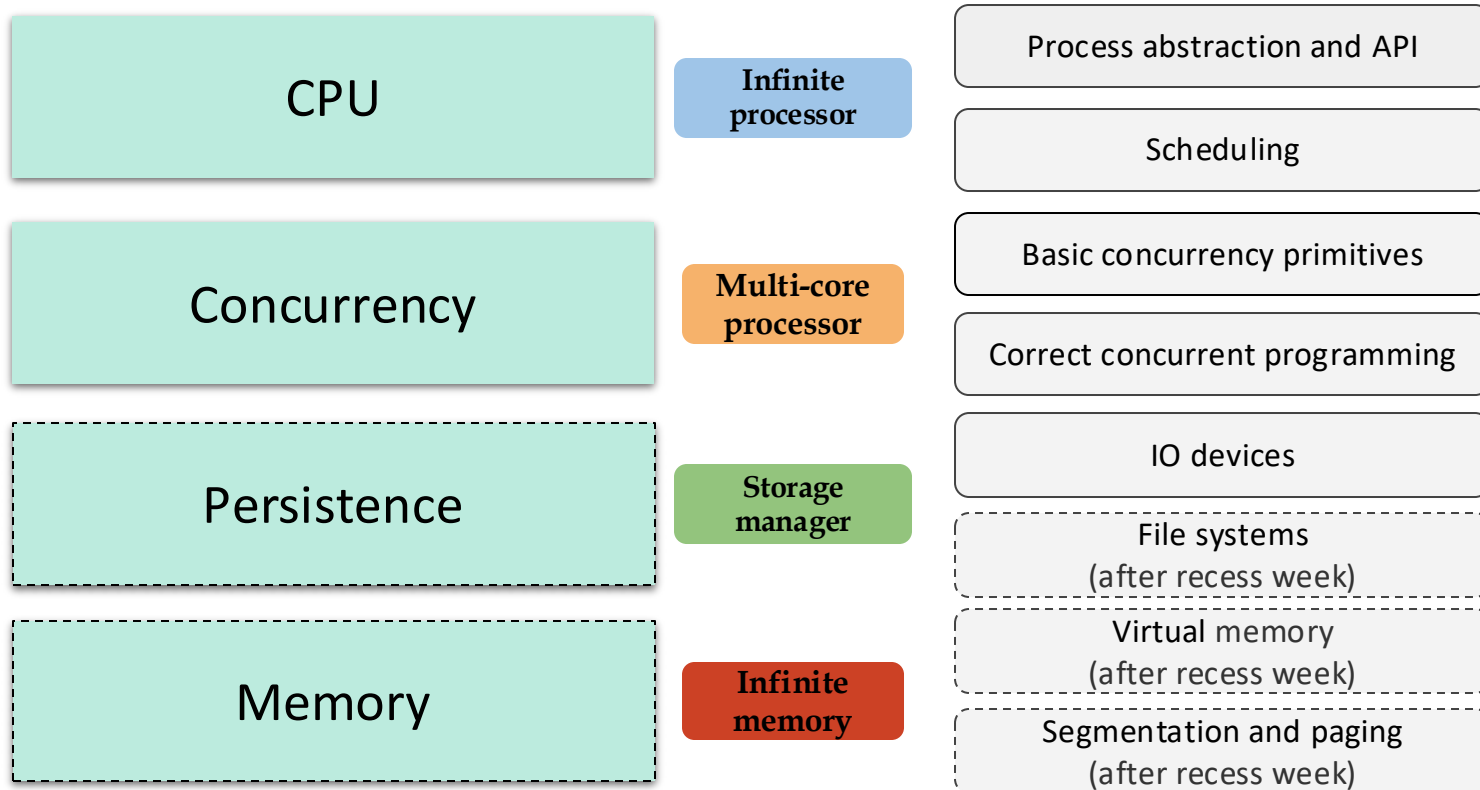
Arvind Easwaran

Cai Wentong

{arvinde, aswtcai}@ntu.edu.sg

This presentation adopts the slides by Prof. Edouard Bugnion (EPFL), the co-founder of VMware

Virtualization Landscape



Outline

Virtualization fundamentals

- Definition
- Multiplexing
- Aggregation
- Emulation

Virtual machines (VMs)

- Definition & history (Popek & Goldberg)
- VMM Construction
 - Virtual CPU



Virtualization Fundamentals

Principle of indirection

Any problem in computer science can be solved by adding a level of indirection. But this usually creates another problem.

David Wheeler

Virtualization applies indirection through **layering** when the layered abstraction is **compatible** with the layer it relies upon

Why CPU virtualization?

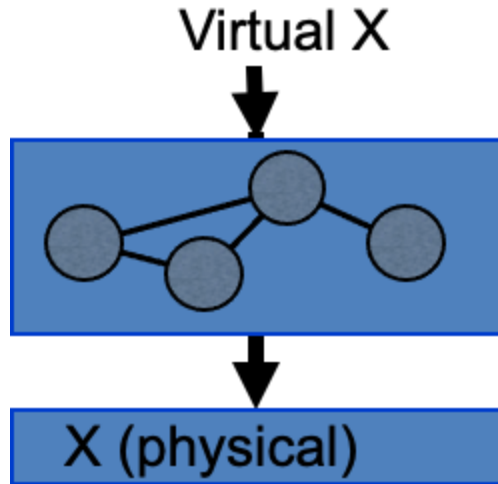
Recall: OS processes share CPU cores

- CPU core can be allocated to a process for a time slot

Reasons for CPU cores virtualization

- To share physical CPU resources among multiple VMs
 - Recall: Multiprogramming improves resource efficiency
- Isolation of workloads for security and performance
 - Example: Cloud servers can run 10s-1000s customers' jobs
- Multiple operating systems to run on the same hardware.

Layered & Compatible



- Layer that exports a compatible abstraction of the layer that it relies on
 - Hiding the physical names from the abstraction to ensure isolation
- Virtualization enforces **modularity**
- Applies to different types of resources
 - Notably CPU, disks, memory

3 main virtualization mechanisms

Virtualization by **multiplexing**

- Expose a single resource among multiple virtual entities
- E.g., CPU, memory

Virtualization by **aggregation**

- Make multiple resources appear as one single virtual resource
- E.g., disk volumes

Virtualization by **emulation**

- Make a physical resource appear as another physical resources
- E.g., a serial port over a USB port, or over SSH

Check Your Understanding



Keep Wooclap open on your device before the lecture ends

Virtualization via Multiplexing

Virtual CPU cores (vCPUs) and scheduling

- Each physical CPU core can run any vCPU
 - **Multiplexing**: The **host** OS schedules vCPUs to runs on CPU cores
 - Example: Linux treats vCPUs as processes
- Each virtual machine's (VM) **guest** OS can schedule its processes on vCPUs
 - Similarly to a non-virtualized OS

Example scenarios

- 4-core server can run 4 single-vCPU VMs
- 4-core server can run 8 2-vCPU VMs (CPU overcommitment)
 - Assumption: Not all VMs have running processes simultaneously



**Aggregation:
many appear as one**

Primary use: aggregating physical resources

Recall from Comp. Arch.: A computer has multiple MMs (memory modules)

- The memory controller aggregates the physical memory into a single (typically contiguous) namespace

A computer has many hard drives

- **RAID** (Redundant Array of Inexpensive Disks) make them appear as a single volume
 - With additional features (see next slide)

A network has multiple links connecting servers

- Link aggregation and multipathing create a virtual link with more bandwidth

RAID: aggregating physical disks into volumes

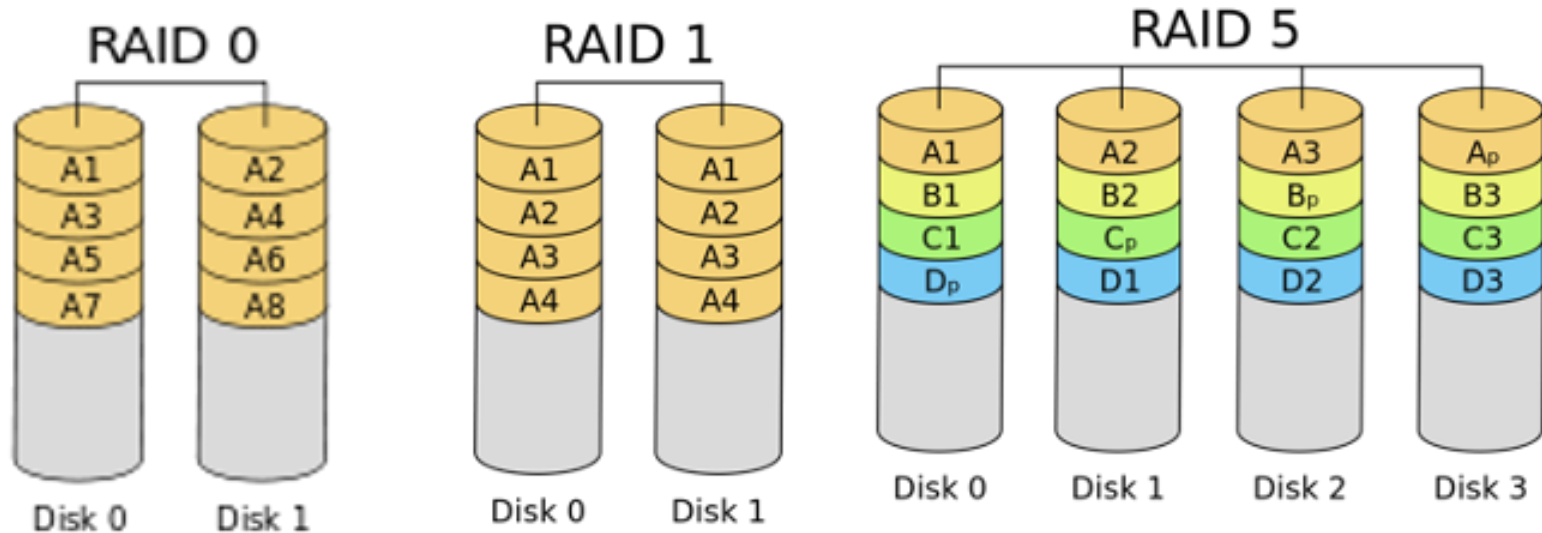
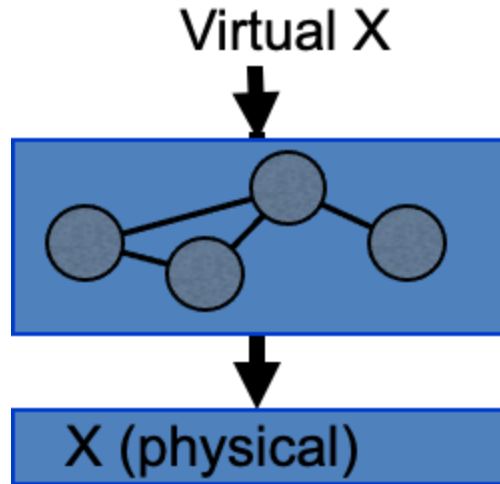


Image credit: wikipedia

Layered & Compatible : volume of sectors

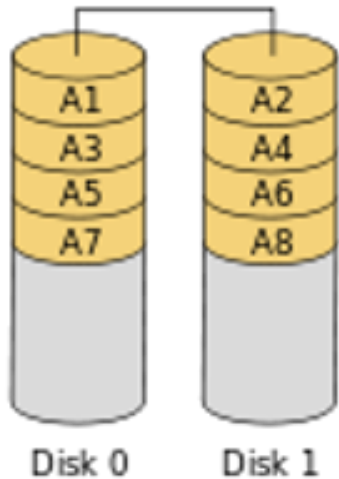


- Aggregate **n** disks into **1** volume
- P = disk with **N** sectors
- V = disk with **M** sectors

RAID: aggregating physical disks into volumes

Striping

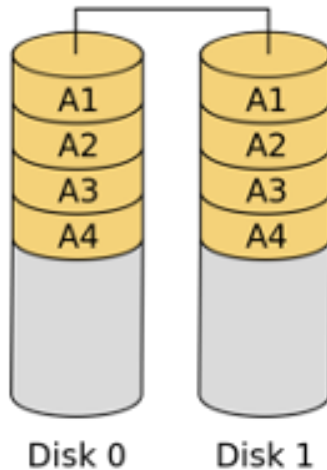
RAID 0



$$M = N * n$$

Mirroring

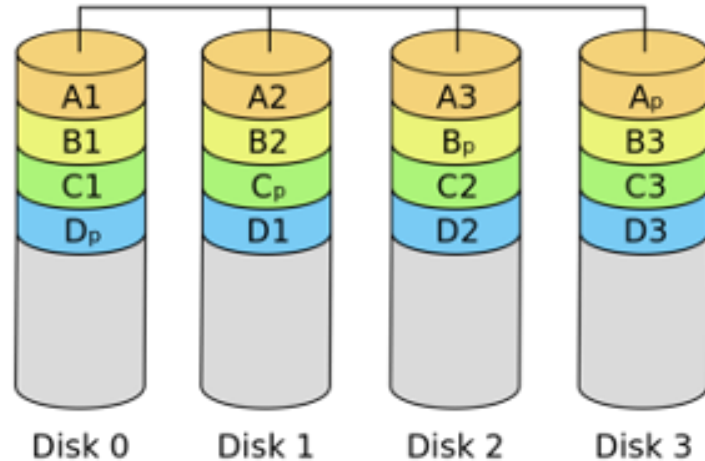
RAID 1



$$M = N; n=2$$

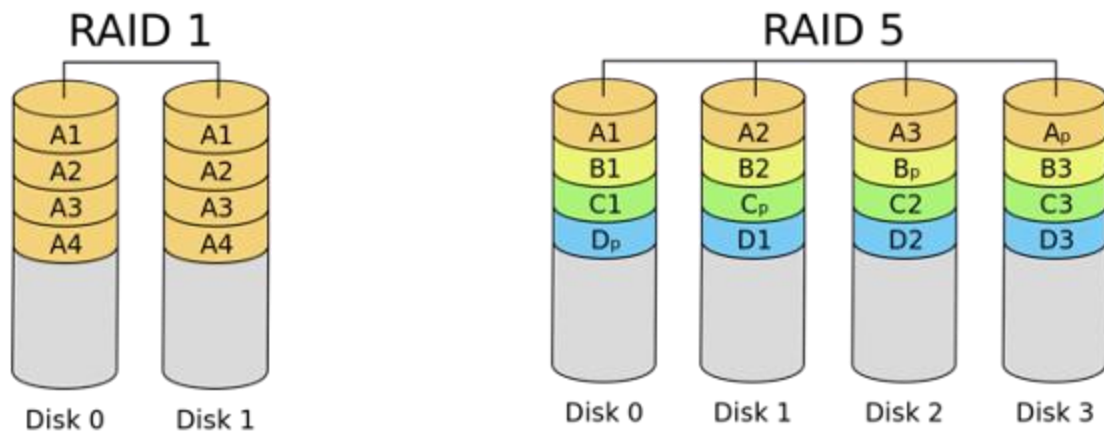
Distributed Parity encoding

RAID 5



$$M = N * (n-1) ; n \geq 3$$

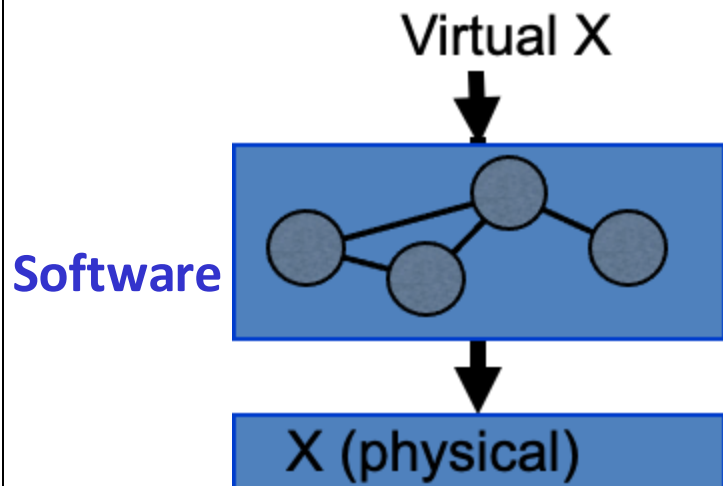
So, which is better ?



A: (hint:) there is nearly always a trade-off in computer systems.

Emulation

Virtualization via **emulation**



- Sometimes there is no physical X, or a different physical X in the same category
 - Category: CPU of a given architecture, block storage, Ethernet NIC, etc.
- Idea: **use software** to emulate a resource different from the underlying resource
- **Expose it within the OS** as a virtual resource

Virtualization via emulation

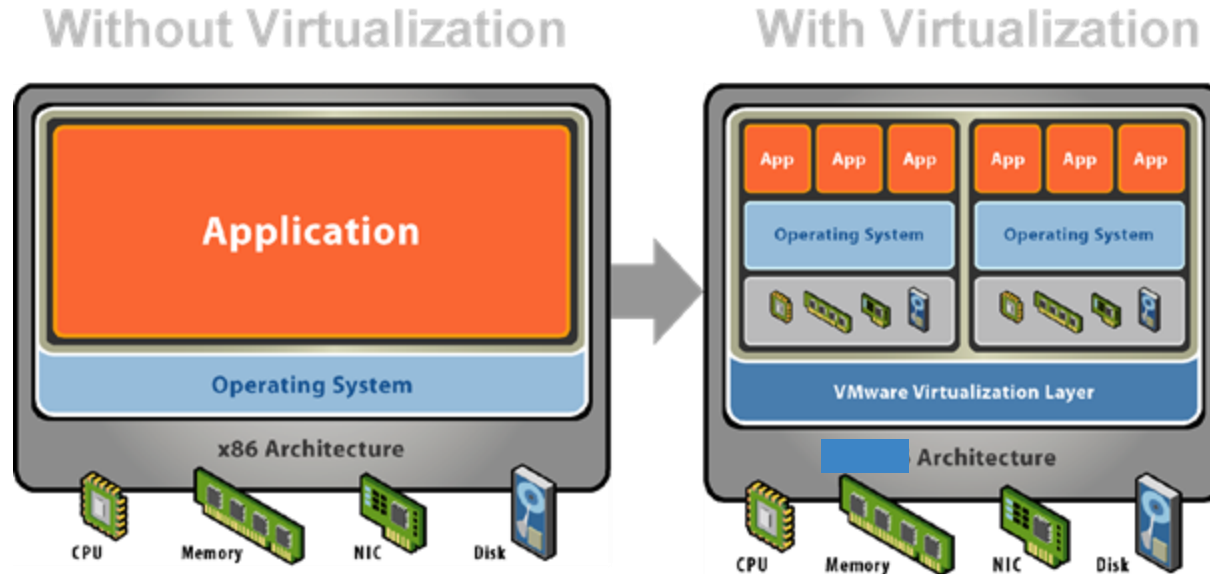
Examples:

- A **RAM disk**: P=DRAM, V=disk to mount a filesystem (/tmp)
- A **Java Virtual Machine**: P=x86-64 or Arm CPU; V=Java byte code processor
- Apple Rosetta - dynamic cross-architectural translation
 - P = x86-64 core; V = PowerPC processor (in 2006)
 - P = Arm v8; V = x86-64 processor (in 2020)
- 127.0.0.1 (localhost interface)
 - P = <none>;
V = a virtual ethernet interface (to if-up IPv4 localhost)



Putting it all together: Virtual Machines

Virtual Machines



Apply virtualization principle to **an entire computer system**

- P = computer system
- V = computer system (== **virtual machine**)

Why virtual machines ?

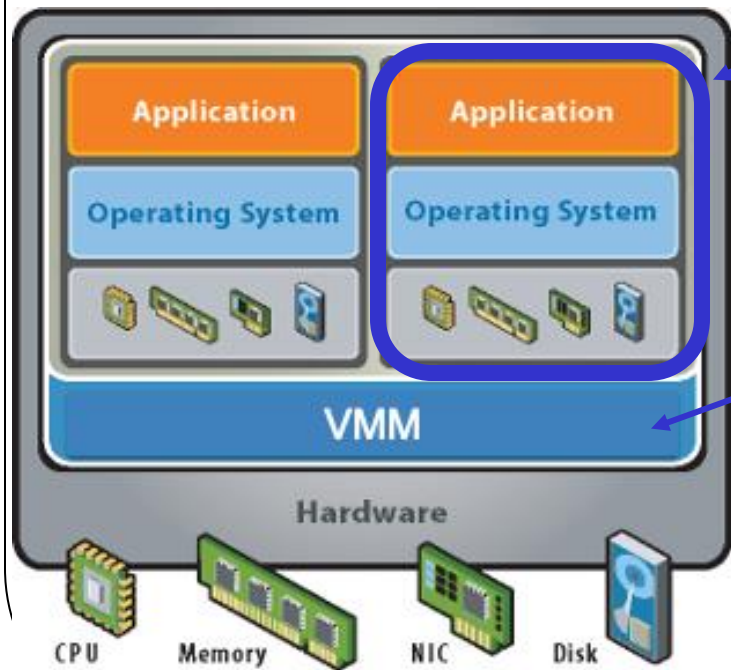
Very popular in the very early days of computing (60's, 70's)

- Hardware was expensive
- Operating systems were primitive (pre-MULTICS)

Back in favor since 2000

- To ensure compatibility with the OS of choice
- To consolidate servers
- As a foundation for multi-tenancy (different owners)
- Because it is easy to provision a VM than a real machine
- **And therefore, as the foundation of the cloud**

Virtual Machines - Def. by Popek and Goldberg [1974]



A virtual machine is an efficient, isolated duplicate of the real machine

The virtual machine monitor (or hypervisor)

- Is always in complete control of the system
- Uses the hardware directly so that programs show at most a minor decrease in speed

Virtual Machines - Terminology

Operating systems

- **Guest operating system** : running inside a virtual machine
- **Host operating system**: running directly on the hardware - combined with the VMM
 - E.g., Linux with KVM

Memory

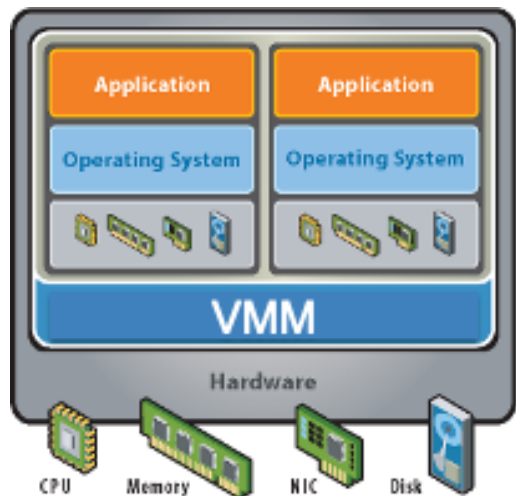
- Virtual memory -- as commonly understood
- **Guest physical memory** -- physical memory of the VM
- **Host physical memory** -- (actual) physical memory

Check Your Understanding



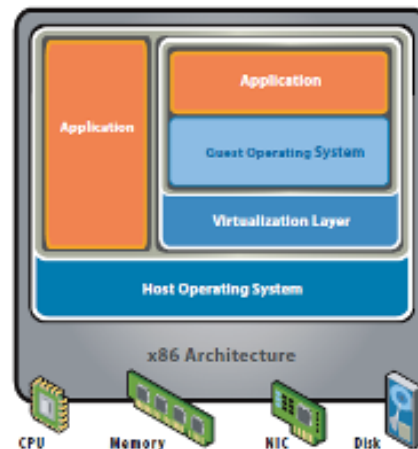
Keep Wooclap open on your device before the lecture ends

Type 1 vs. Type 2



Type 1

- VMM is also the host OS
- E.g., Xen, vSphere



Type 2

- VMM optional extension to the host OS
- E.g. KVM and Linux

Virtualization Levels

1. Full virtualization

- Complete abstraction of the actual hardware
- Guest OS runs unmodified on the hypervisor
 - It is unaware of the hypervisor's existence
- Examples: VMware ESXi, Microsoft Virtual Server

2. Para-virtualization

- Unique software interface (API) between guest OS & VM
- Guest OS needs to be modified to adapt to the new API
- **Advantage:** VM does not virtualize hard-to-implement parts of the H/W instruction set, hence more efficient
- Examples: Xen, VMware, XtratuM

Check Your Understanding



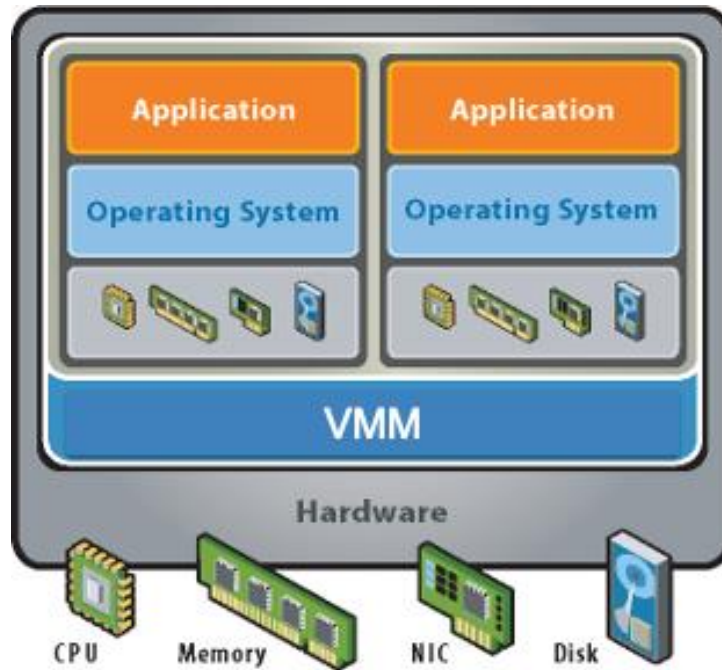
Keep Wooclap open on your device before the lecture ends

Check Your Understanding



Keep Wooclap open on your device before the lecture ends

Constructing a VMM



- P = computer system
- V = computer system (== **virtual machine**)

VMM multiplex and emulate

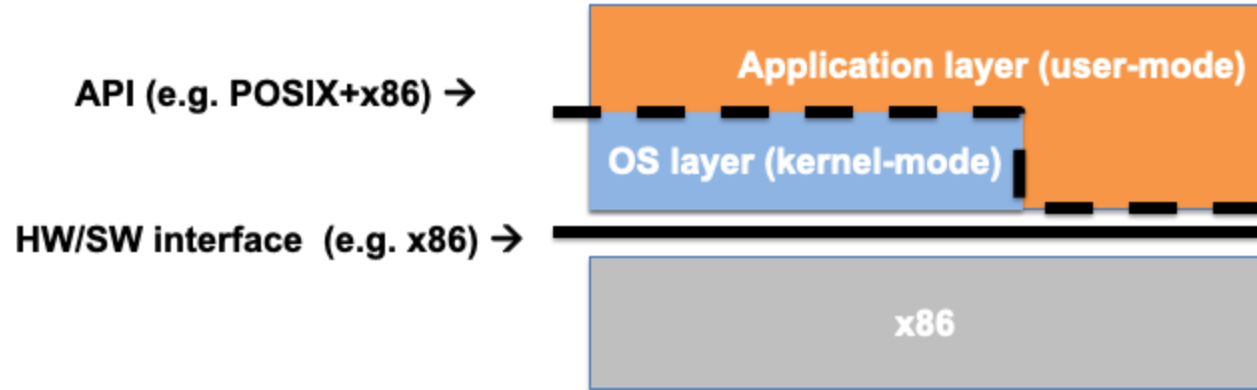
Multiplex CPU and memory, **with hardware support**

- Each VM has its own virtual CPUs (=virtual core)
 - P=core; V=core
 - Requires **system-level limited direct execution**
- Each VM has its own guest physical memory
 - P=physical memory; V=physical memory
 - Requires **hardware support for virtual memory mechanisms**
(details after the recess week)

Emulate I/O devices

- Virtual disk, virtual keyboard, virtual screen, virtual NIC, virtual Ethernet switch, etc.

Bare-metal system -- limited direct execution

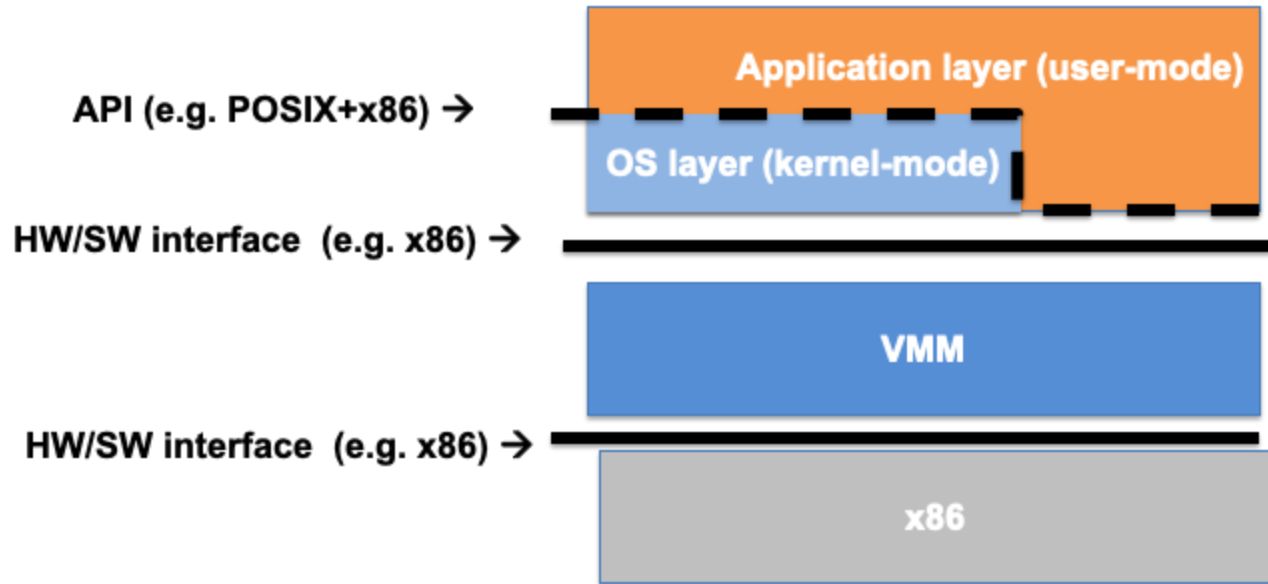


Key observations:

- Application executes unprivileged instructions directly on CPU
- Application make system calls to the OS (--> kernel mode)
- Privileged instructions are restricted to OS (trap from user mode)

This slide is not examinable

Adding a level of indirection

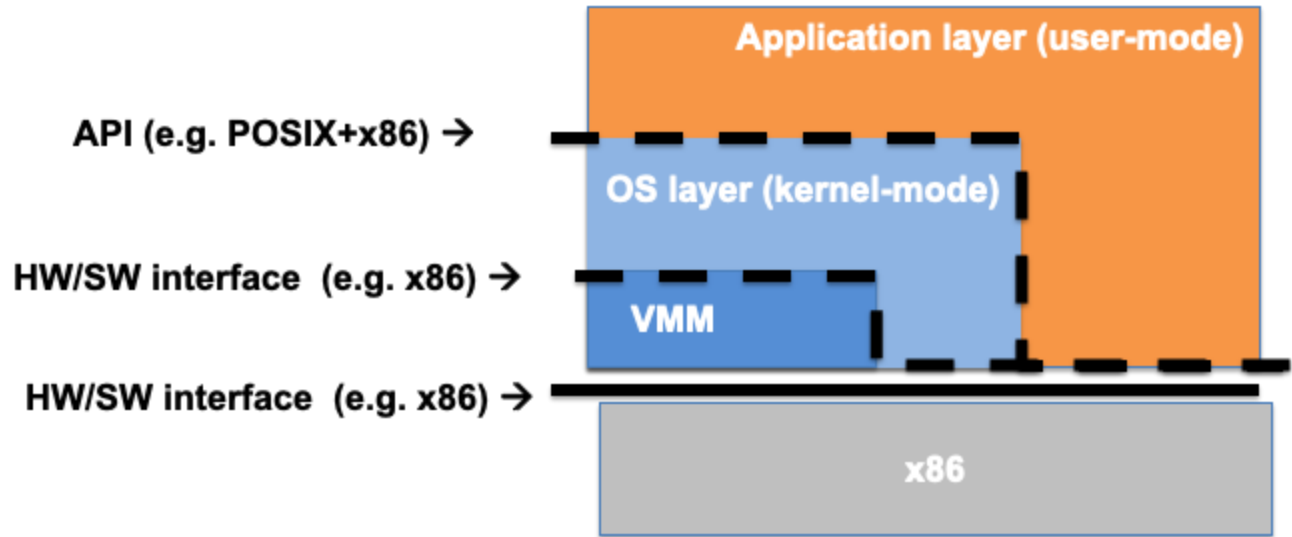


Key observations:

- Virtual CPU == virtualize HW/SW interface (e.g. x86-64)
- Standard virtualization trick -- expose the same abstraction **V** as the underlying resource **P** and everything in between will still work

This slide is not examinable

System-level limited direct execution



Essential for efficiency:

- Application and guest OS use the CPU directly
- Trap (some) privileged instructions issued by guest OS

This slide is not examinable

Virtual CPU -- recap

Multiplexed in space and time -- **very much like kernel threads**

- Space: VMM schedules vCPU from different VMs on different core
- Time: VMM schedules multiple vCPUs on a limited set of physical cores

Isolation guaranteed via:

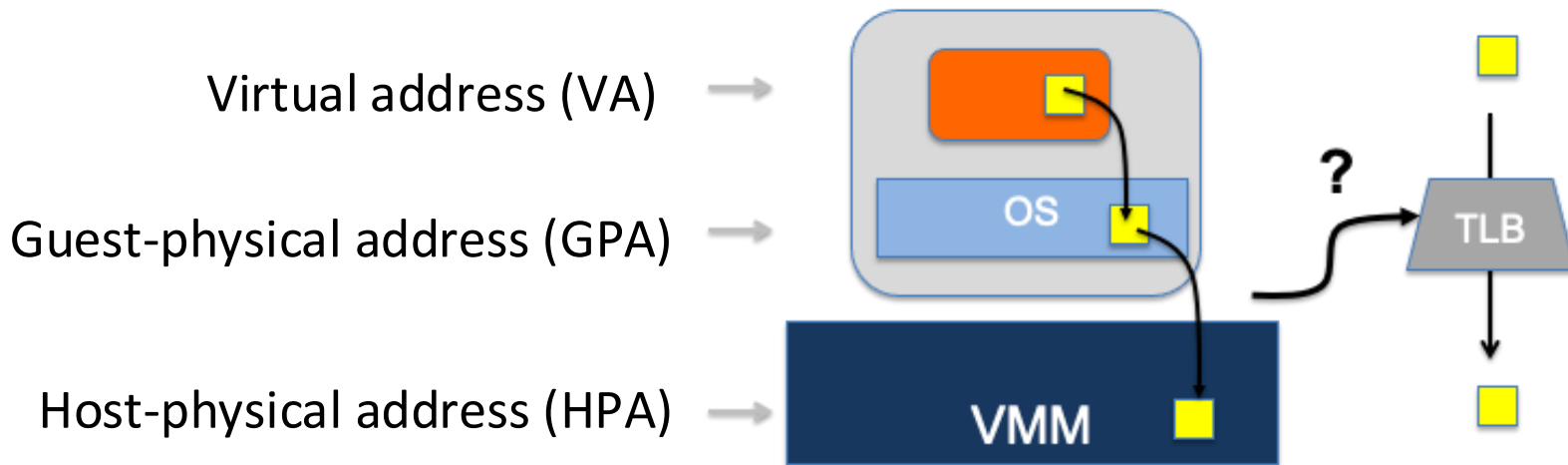
- Non-root mode execution of the VM
- Host OS / VMM is presumed to be correct (**very much like the OS**)

Basic mechanism : limited direct execution

- Same idea as with protected operating system
- **Extended** here **from** application-only **to** system-level

This slide is not examinable

Virtualizing physical memory



Memory address translation

- Processors use VA
- TLB stores mapping between VA and HPA

This slide is not examinable

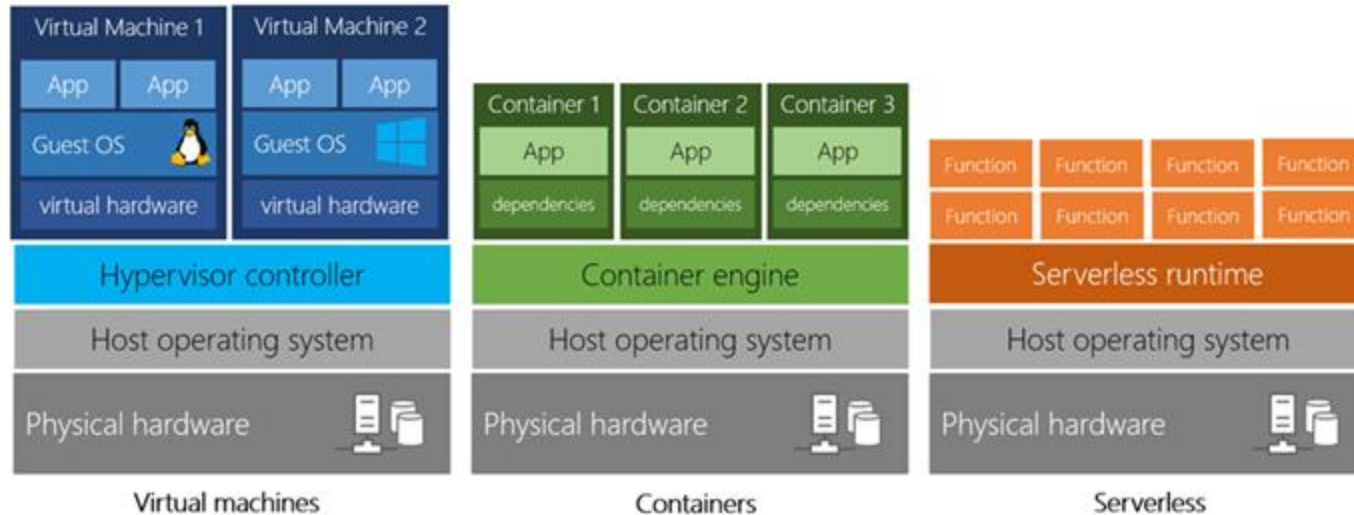
This slide is not examinable

Check Your Understanding



Keep Wooclap open on your device before the lecture ends

Modern Virtualization Landscape



This slide is not examinable

Emerging Virtualization Landscape

Generative AI as a service is booming

- Examples: ChatGPT, Perplexity, Anthropic, Gemini, ...
- Problem: Requires large GPUs clusters -> costly & power hungry

Virtualization is **essential** to make GenAI feasible at scale

- Emulation: Virtual GPU vs. physical GPU
- Multiplexing: GPU memory sharing by many AI models
- Aggregation: Deploy a model on several GPUs at once

This slide is not examinable